

auth.md

How an AI agent signs up for your product on behalf of a user, without a sign-up form, a shared API key, or a bespoke integration.

Published by WorkOS, May 2026 License open, not tied to WorkOS infra Built on OAuth 2.0 + IETF drafts Early adopters Cloudflare, Neon, Resend, Firecrawl, monday.com, Cloudinary

■ User (the human) ■ Agent (Claude, Cursor, a custom bot...) ■ Agent provider (the platform running the agent, e.g. Anthropic / OpenAI) ■ Service (your app: API + authorization server)

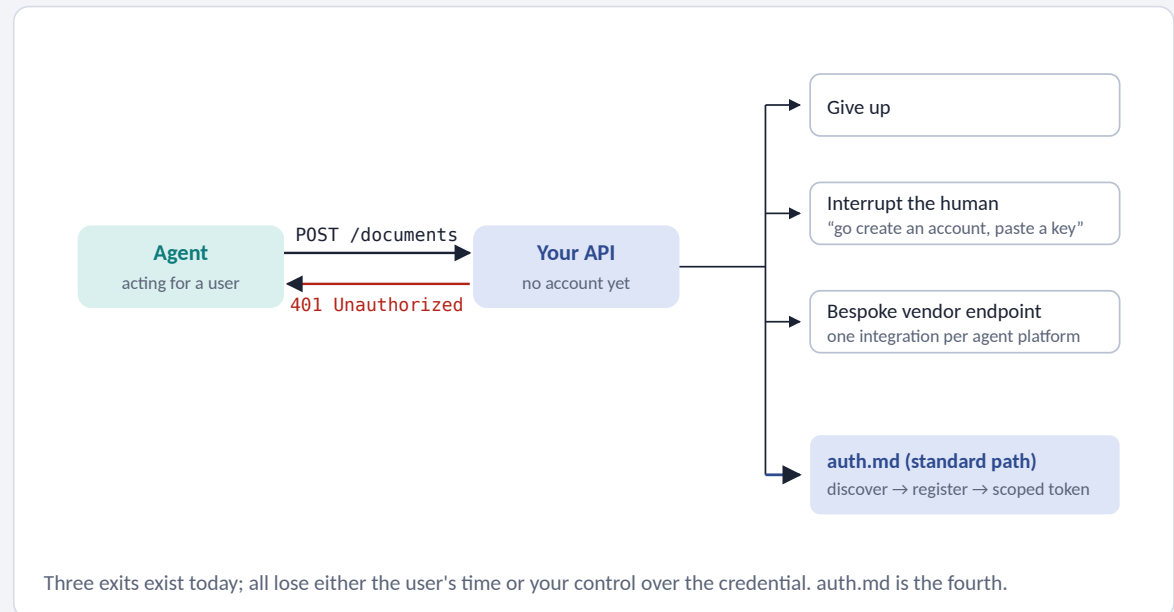
01 The gap it fills: agents have no way to “register on behalf of a user”

Sign-up forms assume a human clicking a screen. Even OAuth assumes a human already has an account and is present at a browser. An agent that hits your API today has three bad exits.

The everyday case: a user asks their agent to “send this contract for signature” or “store this file”. The agent finds your API, calls it, and gets a `401`. From there it can give up, stop the user and ask them to go make an account and paste back an API key, or rely on a one-off endpoint you built just for that agent vendor.

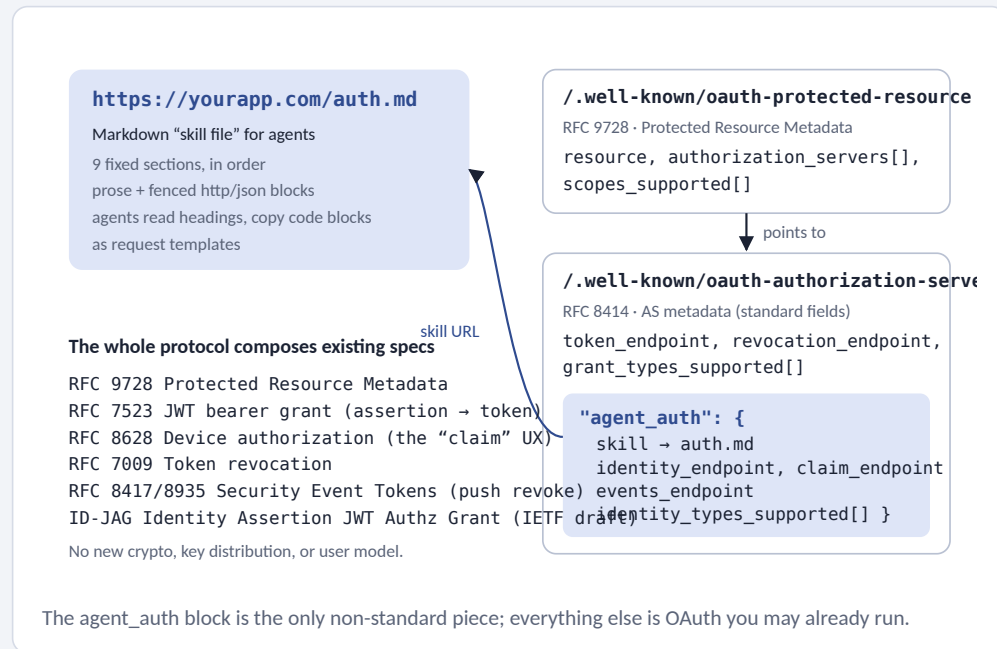
The API-key detour is the one that actually happens in practice, and it is the worst of the three for you: the key is unscoped, not tied to a specific user, shared into the agent's context, and can only be revoked wholesale.

auth.md defines the missing primitive: a standard way for the agent to *discover* how to register, *register* a user-bound identity, and receive a *scoped*, *short-lived*, *revocable* credential.



What it actually is: one Markdown file plus two standard JSON documents

Despite the name, the Markdown file is the human/LLM-readable half. The machine-readable half is ordinary OAuth metadata with one extension block.



Think of it as three layers:

<code>auth.md</code>	A Markdown document at the root of your domain, written <i>to the agent</i> . It says which hostnames are the resource and the authorization server, walks through discovery, tells the agent how to pick a registration method, and gives copy-pasteable request/response shapes.
PRM	Standard Protected Resource Metadata. Tells the agent which authorization server protects the API and which scopes exist.
AS metadata	Standard authorization server metadata plus an <code>agent_auth</code> block listing the registration endpoints, the claim endpoint, the revocation-event receiver, and which identity types you accept.

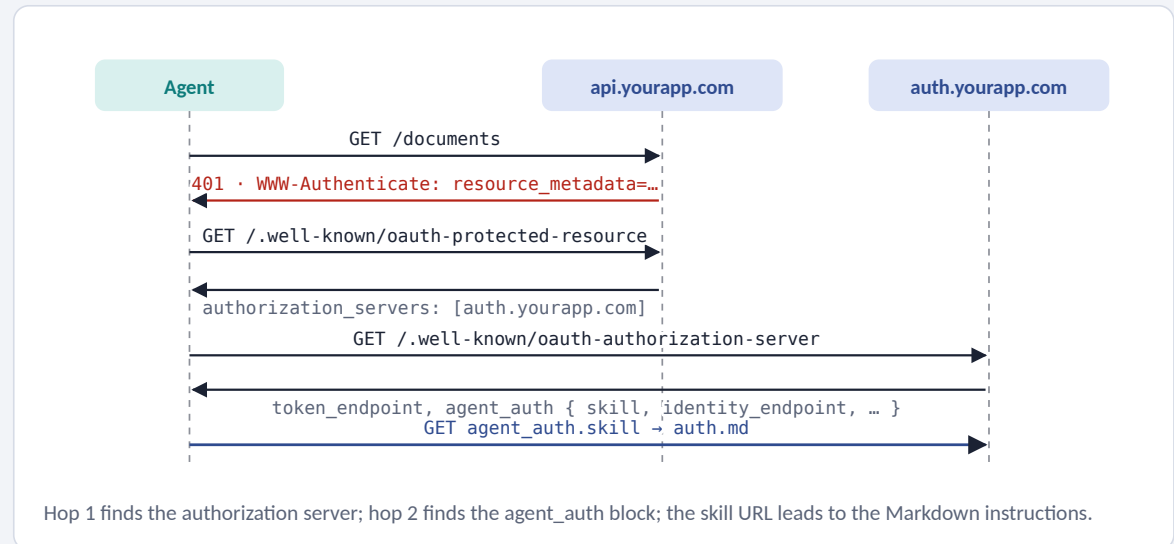
The design bet: agents already read `llms.txt` and `agents.md`, so a Markdown file with fenced code blocks is the most reliably parsed instruction format for an LLM. The JSON metadata is authoritative when the two disagree.

Discovery is a two-hop walk from a 401

The agent never needs to be told about auth.md in advance. A standard `WWW-Authenticate` header on the 401 starts the chain.

- 1 **Call the API** with no credential. The 401 carries `WWW-Authenticate: Bearer resource_metadata=".../.well-known/oauth-protected-resource"`.
- 2 **Fetch PRM**. Read `authorization_servers[]` and `scopes_supported[]`.
- 3 **Fetch AS metadata** from that authorization server. Read the standard OAuth endpoints, then the `agent_auth` block.
- 4 **Fetch `agent_auth.skill`** (the auth.md file) and read it top to bottom: it tells the agent how to choose a method and what to send.

The agent now knows: where to register, which identity types you accept, which grant types your token endpoint speaks, and where revocations will be delivered.



Anatomy of the file: nine sections, always in this order

The spec fixes the section order so an agent can navigate by headings.

Each section maps to one step of the flow.

yourapp.com – agent registration

Intro: "You are an agent. Resource is api..., AS is auth...."

Step 1 – Discover PRM → AS metadata → agent_auth

Step 2 – Pick a method decision tree (slide 05)

Step 3 – Register POST /agent/identity per type

```
```http POST /agent/identity
{ "type": "identity_assertion", "assertion": "..." }
```

**## Step 4 – Claim ceremony** user\_code · verification\_uri · poll

**## Step 5 – Exchange the assertion** token bearer at token\_endpoint

**## Step 6 – Use the access\_token** token bearer header; re-exchange to refresh

**## Errors** code → endpoint → what agent does

```
| login_required | /agent/identity | re-auth, fresh ID-JAG |
```

**## Revocation** RFC 7009 (agent) · SET push (provider)

Fenced code blocks are meant to be lifted verbatim as request templates.

The file is written for a reader that scans headings and copies code blocks — that is, an LLM.

Two things make this different from ordinary API docs.

**It is addressed to the agent.** The intro literally names the resource and authorization-server hostnames and tells the agent what to do first. There is no marketing copy and no assumption of a human reading along.

**It is both prose and data.** The spec's parsing expectations: read the Discovery section first, treat headings as navigation, extract fenced `http / json` blocks as templates, and defer to the JSON metadata whenever the Markdown and the well-known documents conflict.

Practical consequence: your auth.md is generated from the same configuration as your AS metadata, or it drifts. Treat it as a build artefact, not a hand-edited doc.

## Three registration types, chosen by what the agent can prove

The marketing page calls these two “flows” (Agent Verified and User Claimed). The spec exposes three `type` values on `POST /agent/identity`.

TYPE: IDENTITY\_ASSERTION → AGENT VERIFIED

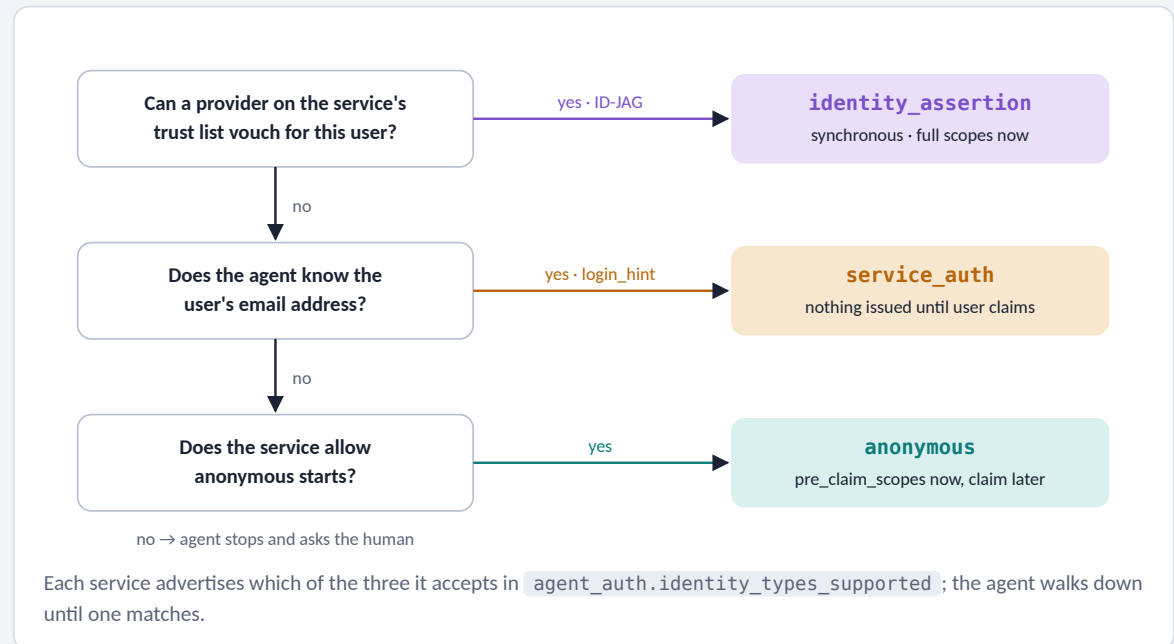
The agent's platform can cryptographically vouch for who the user is (an ID-JAG). Fully synchronous, no human in the loop, credential issued immediately.

TYPE: SERVICE\_AUTH → USER CLAIMED

No provider attestation, but the agent knows the user's email ( `login_hint` ). Nothing usable is issued until the user signs in to *your* app and confirms a 6-digit code.

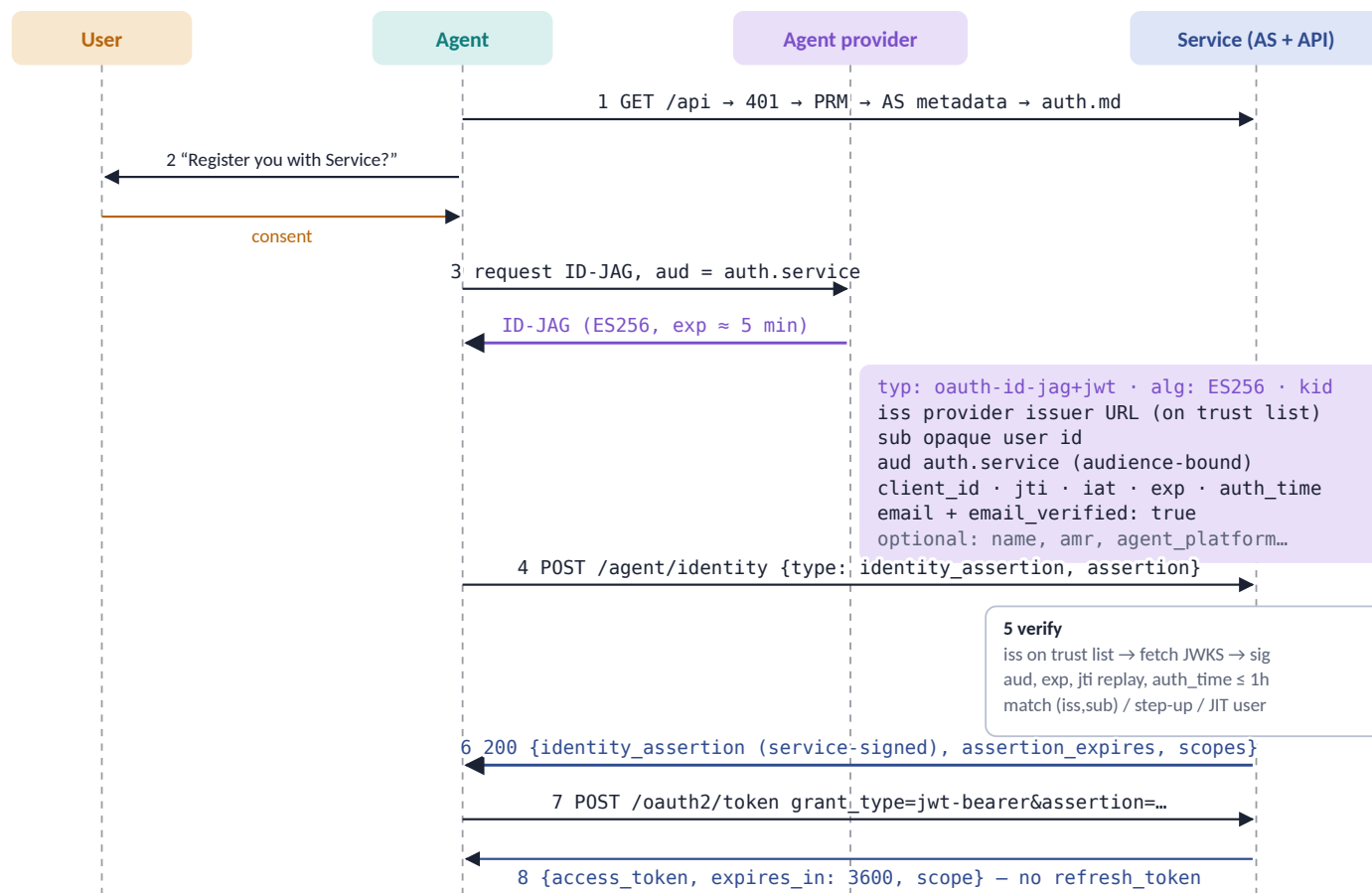
TYPE: ANONYMOUS → USER CLAIMED, DEFERRED

The agent knows nothing. It gets a credential right away with `pre_claim_scopes` only, and can start the claim ceremony later to unlock `post_claim_scopes`.



## Flow A — Agent Verified: the provider vouches, you provision on the spot

This is SSO-style just-in-time provisioning, except the identity provider is the agent's platform and the assertion is an ID-JAG instead of a SAML/OIDC token.



Steps 4 through 6 are the auth.md-specific part. Everything after is a plain RFC 7523 exchange the agent repeats whenever the access token expires.

#### WHY THE SERVICE RE-SIGNS

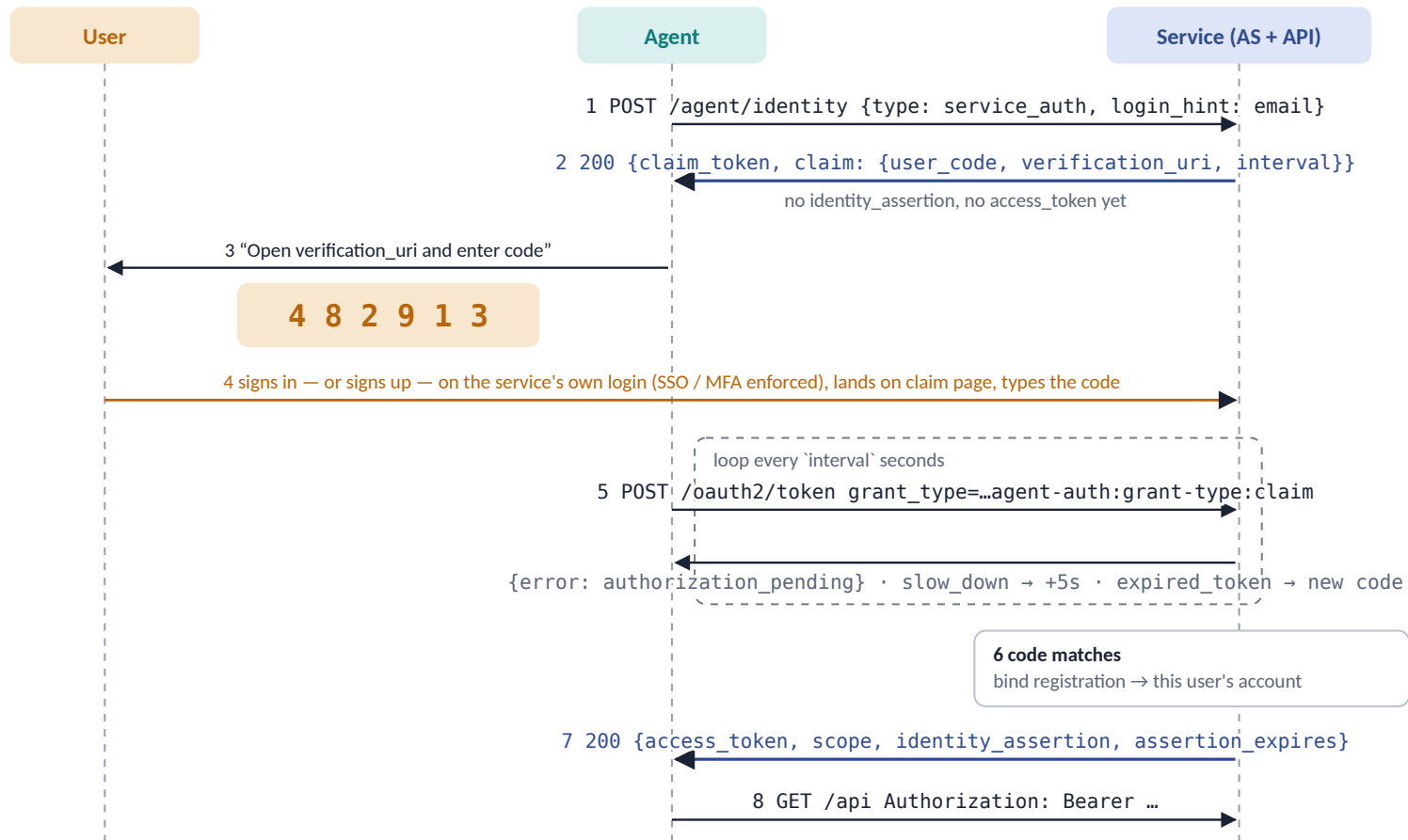
The ID-JAG lives 5 minutes. Your service verifies it once and mints its own `identity_assertion` (also typed `oauth-id-jag+jwt` ) with a longer `assertion_expires` . The agent holds that, not the provider's token, and re-exchanges it for hourly access tokens. Refresh tokens are deliberately never issued.

#### TWO 401S THE AGENT MUST HANDLE

`interaction_required` : a verified email already belongs to an existing account with no delegation from this provider, so the service refuses to bind silently and hands back claim materials (drop into Flow B). `login_required` : `auth_time` is older than the service's max-age, so the agent must re-authenticate the user at the provider with `prompt=login` and get a fresh ID-JAG.

## Flow B — User Claimed: the human proves it on your own sign-in

Mechanically this is RFC 8628 device authorization. The agent is the “device with no browser”; the user finishes the job on your site with your SSO and MFA in force.



The agent never sees the user's password or session. The only thing crossing the gap is a 6-digit code the user reads off one screen and types into another. A user with no account creates one at step 4, on your Sign Up page (slide 08).

#### THE ANONYMOUS VARIANT

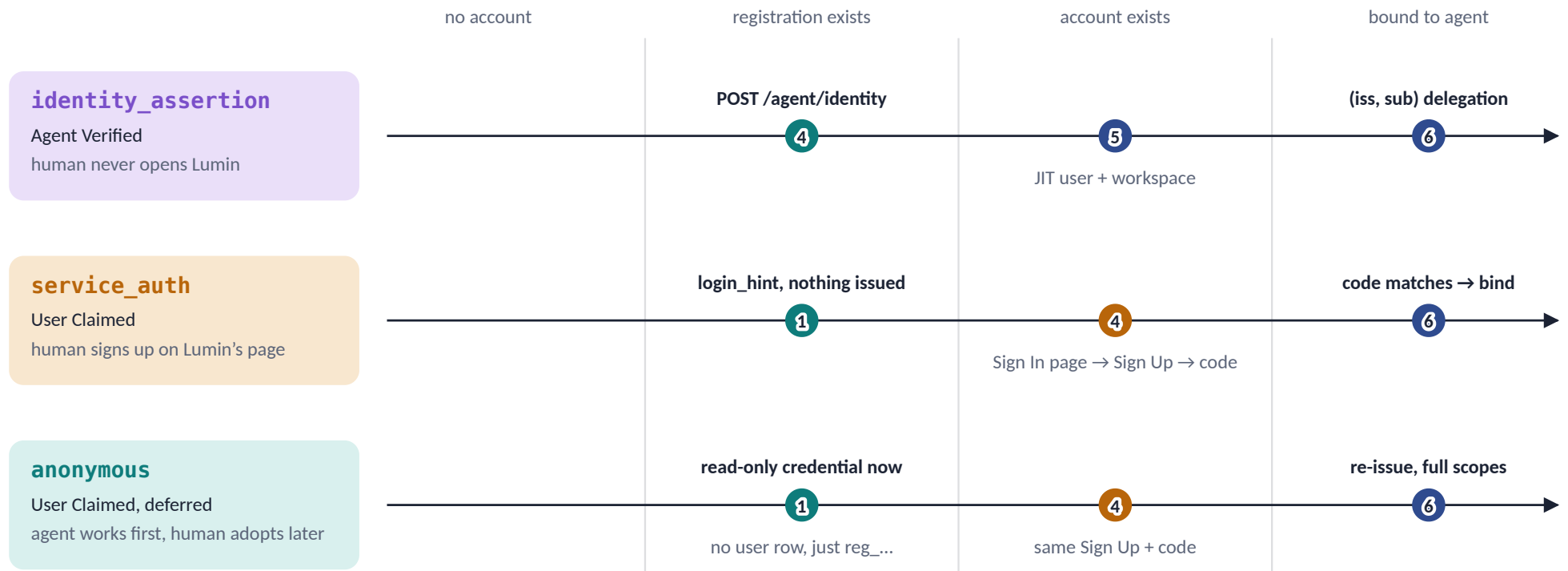
`POST /agent/identity {type: anonymous}` returns an `identity_assertion` immediately, limited to `pre_claim_scopes` (e.g. `api.read`), plus a `claim_token`. When the agent later learns the user's email it calls `POST /agent/identity/claim {claim_token, email}` to mint a `user_code`, and the ceremony above runs. On completion the service revokes the pre-claim tokens and issues a new assertion carrying the email and `post_claim_scopes`.

#### COST OF THIS FLOW

No provider integration, which is why it works for any agent. In exchange, you own a state machine: pre-claim registrations, claim attempts, hashed codes with a  $\leq 10$ -minute TTL, scope transitions, and expiry of unclaimed registrations. The pre-claim credential is real key material, so it must be force-rotatable.

# Where exactly a person goes from “no Lumin account” to “owns one”

The spec never creates an account in the agent's hands. Ownership is established in one of two places: inside your service during ID-JAG verification, or on your own sign-in/sign-up page during the claim ceremony.



Step numbers refer to slides 06 and 07. The account-creating moment is column three; only the top row reaches it without a human touching Lumin.

#### ANSWERING THE QUESTION DIRECTLY

Yes. In the User Claimed flow, step 4 means the person lands on Lumin's own Sign In page (the one in your screenshot). A brand-new user clicks **Sign Up** or picks Google/Apple/Microsoft/Passkey, creates the account under Lumin's normal rules, and is then forwarded to the claim page to type the 6-digit code. The spec only says “authenticates through your sign-in flow”; whether that flow includes sign-up, which identity providers are offered, T&C acceptance, and data region are all Lumin's decisions, which is the point of the design.

#### THE ONE PATH THAT SKIPS THE PAGE

In Agent Verified, the account is created server-side at step 5: Lumin trusts the provider's `email_verified: true` and JIT-provisions a user, exactly like SAML JIT from an enterprise IdP. No password ever exists; the user later “adopts” the account by signing in with the same email via Google/passkey or a reset link. If that email already belongs to a Lumin account with no delegation from this provider, the spec requires a step-up: Lumin returns `interaction_required` and the user proves ownership through the claim ceremony instead.

## Worked example: Claude, Anthropic, and a Lumin Sign request

Son, who has no Lumin account, tells Claude: “Send *MSA-v3.pdf* to Alice for signature.” Below is what would cross the wire if Lumin implemented `auth.md`.

**What is real and what is invented.** Real, from developers.luminpdf.com:

`api.luminpdf.com/v1` , `auth.luminpdf.com/oauth2/token` , the endpoint `POST /v1/signature_request/send` and its body, and the scopes `sign:requests` , `pdf:files` , `profile.read` . Invented for illustration: Anthropic as an ID-JAG issuer and its hostname, Lumin's `/agent/*` endpoints and `auth.md` (Lumin has not published one), and every token, id and code shown.

### Path A • Anthropic is on Lumin's trusted-provider list (Agent Verified)

**CLAUDE** A1

#### Tries the API cold

Claude's Lumin tool has no credential for Son yet.

```
POST https://api.luminpdf.com/v1/signature_request/send
Content-Type: application/json
{ "title": "MSA v3 - Acme", "file_url": "...", "signers": [{ "email_address": "alice@acme.example", "name": "Alice" }], "expires_at": 1760572

← 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata="https://api.luminpdf.com/.well-known/oauth-protected-resource"
```

CLAUDE A2

### Discovers, two hops

PRM names the authorization server;

AS metadata carries the

agent\_auth block; the skill file explains the rest.

```
GET https://api.luminpdf.com/.well-known/oauth-protected-resource
← { "resource": "https://api.luminpdf.com/v1", "authorization_servers": ["https://auth.luminpdf.com"],
 "scopes_supported": ["sign:requests", "sign:requests.read", "pdf:files", "pdf:files.read", "templates", "agreements", "profile.read"] }

GET https://auth.luminpdf.com/.well-known/oauth-authorization-server
← { "issuer": "https://auth.luminpdf.com",
 "token_endpoint": "https://auth.luminpdf.com/oauth2/token",
 "revocation_endpoint": "https://auth.luminpdf.com/oauth2/revoke",
 "grant_types_supported": ["urn:ietf:params:oauth:grant-type:jwt-bearer", "urn:workos:agent-auth:grant-type:claim"],
 "agent_auth": {
 "skill": "https://www.luminpdf.com/auth.md",
 "identity_endpoint": "https://auth.luminpdf.com/agent/identity",
 "claim_endpoint": "https://auth.luminpdf.com/agent/identity/claim",
 "events_endpoint": "https://auth.luminpdf.com/agent/event/notify",
 "identity_types_supported": ["identity_assertion", "service_auth"], ← anonymous deliberately off for Sign
 "identity_assertion": { "assertion_types_supported": ["urn:ietf:params:oauth:token-type:id-jag"] } } }

GET https://www.luminpdf.com/auth.md ← Markdown; Claude reads “Pick a method” and “Register”
```

SON A3

### Consents inside Claude

Claude: “Lumin needs an account for you. May I register *son@example.com* with Lumin using your Anthropic identity, with permission to create signature requests and upload files?” Son: “Yes.”

requested scopes: sign:requests pdf:files profile.read

ANTHROPIC A4

### Mints the ID-JAG

Claude asks its platform for an assertion audience-bound to Lumin's authorization server. Five-minute lifetime, signed with Anthropic's key.

```
eyJhbGciOiJIJFuzI1NiIsInR5cCI6IWRwLWphZytqd3QiLCJraWQiOiJhbnRoLWlkamFnLTlwMjYtMDkifQ
.eyJpc3MiOiJodHRwczovL2FldGguYW50aHJvcGljLmNvbSIsInN1YiI6InVzcl8wMU05S1RaWTd0M1JXNVF2VjdIU1FRRzhBMjYsImF1ZCI6Imh0dHBz0i8vYXV0aC5sdWlpbmBkZi5i
.MEUCIQD6...(signature)...
```

```
decoded header { "alg": "ES256", "typ": "oauth-id-jag+jwt", "kid": "anth-idjag-2026-09" }
decoded payload { "iss": "https://auth.anthropic.com",
 "sub": "usr_01J9KTZY7N3RW5QvV7HSQQG8A3", ← opaque, stable per user
 "aud": "https://auth.luminpdf.com", ← usable only at Lumin
 "client_id": "https://auth.anthropic.com",
 "jti": "3f214001-6a57-4852-99f2-35a26cb28e6d",
 "iat": 1758008000, "exp": 1758008300, ← 5 minutes
 "auth_time": 1758006900, ← Son signed in 18 min ago
 "email": "son@example.com", "email_verified": true, "name": "Son",
 "amr": ["pwd", "mfa"],
 "agent_platform": "claude.ai", "agent_context_id": "conv_01J9KTZABC" }
```

CLAUDE A5

## Registers with Lumin

```
POST https://auth.luminpdf.com/agent/identity
{ "type": "identity_assertion",
 "assertion_type": "urn:ietf:params:oauth:token-type:id-jag",
 "assertion": "eyJhbGciOiJIJFZlI1NiIs..." }
```

LUMIN A6

## Verifies, then creates the account

This is the moment Son goes from no account to owning one. Nothing was typed on luminpdf.com.

```
trust list iss https://auth.anthropic.com → display "Anthropic (Claude)", policy: requires mfa in amr ✓
JWKS GET https://auth.anthropic.com/.well-known/jwks.json → key anth-idjag-2026-09 → signature ✓
claims aud == https://auth.luminpdf.com ✓ exp in future ✓ jti unseen ✓ auth_time 18 min < 60 min ✓ email_verified ✓
match delegation (iss, sub)? none. user with verified email son@example.com? none.
→ JIT create user usr_lmn_8f3a21 + personal workspace ws_lmn_c0d1e2 (region per Lumin policy)
→ delegation store (https://auth.anthropic.com, usr_01J9KTZY...) → usr_lmn_8f3a21
→ mint identity_assertion typ oauth-id-jag+jwt, signed by auth.luminpdf.com, valid 30 days

← 200 { "registration_id": "reg_lmn_01K5A7...",
 "registration_type": "identity_assertion",
 "identity_assertion": "eyJhbGciOiJIJFZlI1NiIsImtpZCI6Imx1bWluLWFzLTlWmjYifQ...",
 "assertion_expires": "2026-10-16T09:00:00.000Z",
 "scopes": ["sign:requests", "pdf:files", "profile.read"] }
```

CLAUDE A7

## Exchanges for an access token

Same token endpoint Lumin's OAuth apps already use. Repeated every hour; no refresh token.

```
POST https://auth.luminpdf.com/oauth2/token
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&assertion=eyJhbGciOiJIJFZlI1NiIsImtpZCI6Imx1bWluLWFzLTlWmjYifQ...
&resource=https://api.luminpdf.com/v1

← 200 { "access_token": "lmn_at_9Qz...", "token_type": "Bearer", "expires_in": 3600,
 "scope": "sign:requests pdf:files profile.read" }
```

CLAUDE A8

## Retries the original call — it works

```
POST https://api.luminpdf.com/v1/signature_request/send
Authorization: Bearer lmn_at_9Qz...
{ "title": "MSA v3 - Acme", "file_url": "...", "signers": [{ "email_address": "alice@acme.example", "name": "Alice" }], "expires_at": 1760572...

← 201 { "signature_request": { "signature_request_id": "sr_01K5A8...", "created_at": "2026-09-16T09:02:11Z", "status": "WAITING_FOR_PROCESSING"

Claude → Son: "Sent. Alice will get the signing email from Lumin. Your new Lumin account is son@example.com - sign in any time with Google or
```

**SON** A9

### Adopts the account, later

Opens luminpdf.com, chooses Google (same email), and finds the MSA request already in his workspace. Settings show “Connected agents: Anthropic (Claude) — revoke”. Revoking there, or disconnecting Lumin inside Claude, fires the SET on slide 10.

```
GET https://api.luminpdf.com/v1/user Authorization: Bearer lmn_at_9Qz...
← 200 { "id": "usr_lmn_8f3a21", "email": "son@example.com", ... }
```

### Path B • Anthropic is *not* on Lumin's trust list (User Claimed, service\_auth)

Realistic for the near term: no large agent platform is a published ID-JAG issuer yet. Steps A1–A3 are identical; the fork happens at registration.

**CLAUDE** B4

### ID-JAG rejected, falls back

```
POST https://auth.luminpdf.com/agent/identity { "type": "identity_assertion", ... }
← 400 { "error": "issuer_not_enabled" }

POST https://auth.luminpdf.com/agent/identity
{ "type": "service_auth", "login_hint": "son@example.com" }
← 200 { "registration_id": "reg_lmn_01K5B2...", "registration_type": "service_auth",
 "claim_token": "clm_7Hd2...", ← Claude keeps this, hashed at Lumin
 "claim_token_expires": "2026-09-16T09:15:00Z",
 "post_claim_scopes": ["sign:requests","pdf:files","profile.read"],
 "claim": { "user_code": "482913", "expires_in": 600, "interval": 5,
 "verification_uri": "https://auth.luminpdf.com/agent/claim" } }
 no identity_assertion, no access_token
```

**CLAUDE** B5

### Hands off to Son

Claude → Son: “Lumin needs you to confirm it's really you.  
Open <https://auth.luminpdf.com/agent/claim> and enter code 482 913.  
It expires in 10 minutes. I'll continue as soon as you're done.”

```
meanwhile, every 5 s:
POST https://auth.luminpdf.com/oauth2/token
grant_type=urn:workos:agent-auth:grant-type:claim&claim_token=clm_7Hd2...
← 400 { "error": "authorization_pending" }
```

**SON**

B6

### Creates the account on Lumin's page

The claim URL sits behind Lumin auth, so Son sees the Sign In page on the right. He has no account, so he clicks **Sign Up** (or Google). Lumin's normal onboarding runs: verification, T&Cs, workspace creation. Then he is returned to the claim page and types **482 913**.

```
Lumin: code hash matches reg_lmn_01K5B2... login_hint son@example.com == signed-in user's email ✓
→ bind registration → usr_lmn_8f3a21 (just created)
→ mint identity_assertion, scopes = post_claim_scopes
```

### Sign In

Recommend work email but not required.

  
Google

  
Apple

  
Microsoft

  
Dropbox

  
Xero

  
Passkey

OR USE YOUR EMAIL

 Enter your email

 Enter your password 

Forgot Password?

Sign In

New to Lumin? **Sign Up**

[Switch to classic view](#)

What Son sees at step B6 when he opens `verification_uri` with no account: Lumin's own Sign In page. "New to Lumin? Sign Up" is the account-creation moment; the claim page with the code field comes right after.

**CLAUDE**

B7

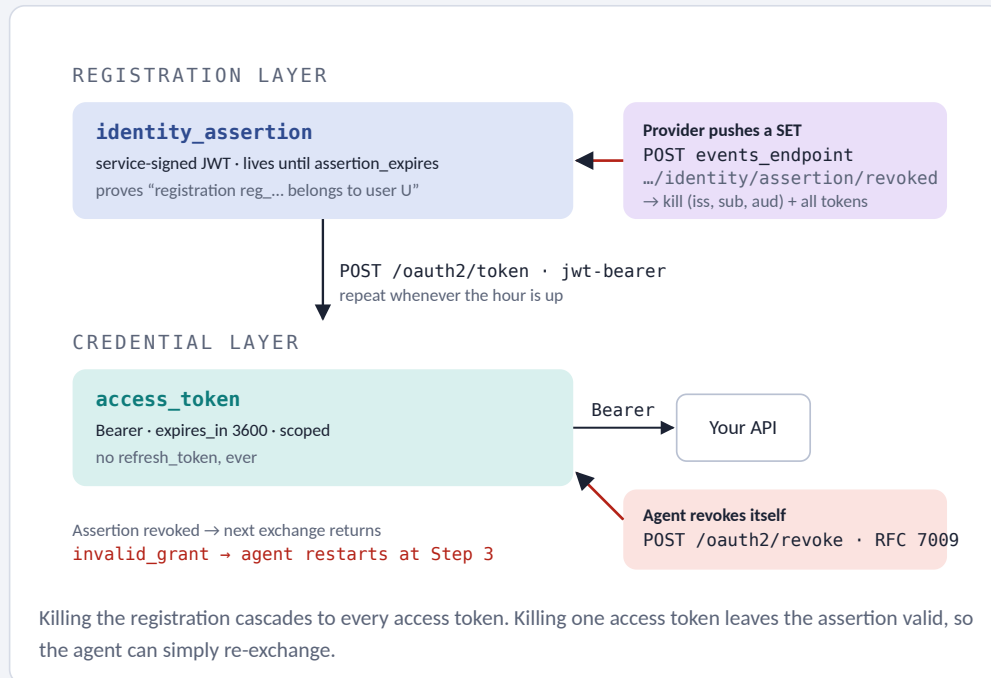
### Next poll succeeds; continues as A8

```
POST https://auth.luminpdf.com/oauth2/token (same claim grant)
← 200 { "access_token": "lmn_at_2Vx...", "token_type": "Bearer", "expires_in": 3600,
 "scope": "sign:requests pdf:files profile.read",
 "identity_assertion": "eyJhbGciOiJIUzI1NiIsImtpZCI6Imx1bWluLWFzLTIwMjYifQ...",
 "assertion_expires": "2026-10-16T09:07:40.000Z" }

POST https://api.luminpdf.com/v1/signature_request/send Authorization: Bearer lmn_at_2Vx...
← 201 { "signature_request": { "signature_request_id": "sr_01K5B9...", "status": "WAITING_FOR_PROCESSING" } }
```

# The credential model: two tokens, two revocation layers

Whichever flow got you there, the agent ends up holding the same pair, and each has its own way to die.



## Why no refresh token?

The `identity_assertion` already plays that role, but with better properties: it is bound to a registration record the service controls, it is the unit that provider-initiated revocation targets, and re-exchanging it forces every hour of access through your token endpoint where policy is re-evaluated.

## Provider-initiated revocation

If a user disconnects the agent platform, or the platform detects compromise, the provider sends a Security Event Token to your advertised `events_endpoint`. You verify it against the provider's JWKS exactly as you verified the ID-JAG, check `jti` for replay, and invalidate everything keyed on (iss, sub, aud). Return `202`.

For a signing product this is the property that matters: a delegated agent identity can be cut off from the platform side, per user, without touching anyone else's access.

# What the service side has to build

If you already run an OAuth authorization server, most of this is configuration plus one new resource. If you don't, the token endpoint is the real work.

## ENDPOINTS (5)

POST `/agent/identity` — dispatch on `type`  
 POST `/oauth2/token` — jwt-bearer + claim grant  
 POST `/oauth2/revoke` — RFC 7009  
 POST `/agent/identity/claim` — anonymous only  
 POST `/agent/event/notify` — SET receiver

`/.well-known/oauth-protected-resource`

`/.well-known/oauth-authorization-server`

`/auth.md`

## TRUST & IDENTITY LOGIC

- **Trusted-provider allow-list:** issuer URL, your own display name (never render the provider's `client_name`), optional JWKS URI and attestation policy such as “requires `mfa` in `amr`”.
- **User matching order:** existing `(iss, sub)` delegation → same verified email/phone but no delegation → *step-up*, never silent bind → no match → JIT-provision or refuse.
- **Claim verification:** `aud`, `exp`, `iat` skew, `jti` replay cache, `auth_time` ≤ 1h, at least one of `email_verified` / `phone_number_verified`.

## ABUSE CONTROLS THE SPEC PRESCRIBES

- Sliding-window rate limits, per-IP checked first: **5/h anonymous**, 60/h `identity_assertion` per IP; 100/h and 1000/h per tenant.
- SHA-256 hash `claim_token`, `claim_attempt_token`, `user_code`; return plaintext once.
- CSPRNG `user_code`, TTL ≤ 10 min. JWKS cache floor 10 min, ceiling 24 h.
- Log IPs at register, claim, complete. Operator-facing bulk revocation for incidents.
- Audit events: `registration.created`, `assertion.issued`, `token.issued`, `claim.confirmed`, `registration.revoked` ...

12

# Where it sits next to things you already run

auth.md does not replace MCP authorization or your OAuth login. It handles the step before them: getting an account and a delegation to exist at all.

MECHANISM	WHO IS BEING IDENTIFIED	NEEDS AN EXISTING ACCOUNT?	HUMAN PRESENT?	WHAT COMES OUT
Shared API key	Nobody in particular	Yes, someone made it	Once, to copy the key	Unscoped, un-attributable, revoke-all-or-nothing
OAuth 2.1 authorization code (what MCP authorization uses)	A user with a session	Yes	Yes, at a browser	Access + refresh token for a registered client
Dynamic Client Registration (RFC 7591)	The client software	Not for the client; still needs a user later	No	<code>client_id</code> , not a user identity
auth.md • Agent Verified	The user, attested by the agent's platform	No — JIT-provisioned	Only for consent inside the agent	User-bound assertion → hourly scoped tokens
auth.md • User Claimed	The user, proven on your login	Created or matched during the claim	Yes, briefly, on your site	User-bound assertion → hourly scoped tokens

Natural pairing: an agent uses auth.md to provision the account and obtain the first credential, then your MCP server's normal OAuth protection keeps working unchanged because the token endpoint is the same authorization server.

# Reading it through Lumin's lens

The “agent signs up, user adopts the account later” model you've been sketching is, almost exactly, the spec's `anonymous` + claim path. Some things worth weighing rather than a recommendation.

## Where it maps cleanly

**Two-phase adoption** is native: `pre_claim_scopes` gates what an unclaimed agent can do; the claim ceremony runs on Lumin's own sign-in with SSO/MFA; the post-claim assertion re-issues with the user's email. Nothing bespoke to design there.

**Cross-product:** one authorization server can front Lumin PDF, Lumin Sign and AgreementGen as separate protected resources, each with its own PRM and scope list, sharing the trust list and the registration store.

**Sign's exposure:** the spec's own controls line up with the phishing concern — anonymous registrations at 5/hour per IP, sending or signing scopes kept strictly `post_claim`, hashed one-shot codes, and the step-up rule that prevents an ID-JAG from silently attaching to an existing account that shares a verified email.

## Open questions the spec leaves to you

- **Which providers to trust.** Agent Verified only works for issuers on your allow-list; the spec is silent on who those should be and none of the large agent platforms is named as a live ID-JAG issuer in the public material.
- **Provisioning policy.** JIT-creating a Lumin account from an ID-JAG with only a verified email is a product decision (free tier? workspace? region for data residency?), not a protocol one.
- **What an anonymous agent may do at all.** `pre_claim_scopes` could plausibly be read-only, or empty for Sign. The spec allows you to disable `anonymous` entirely per service.
- **Maturity.** Released May 2026; adopters are infrastructure-flavoured companies. ID-JAG is still an IETF draft. The WorkOS repo ships a reference service and provider as a pnpm monorepo.

The rate limits and hashing rules live in the “For apps” implementation guide as guidance with example values, not as stated conformance requirements. Anything you ship for Sign should treat them as a floor.

# Sources

Everything above is drawn from the following, read on 16 September 2026. Field names, endpoints, error codes and limits are quoted from the spec and the “For apps” implementation guide.

- [workos.com/auth-md](https://workos.com/auth-md) — overview, flows, adopters
- [workos.com/auth-md/docs](https://workos.com/auth-md/docs) — documentation index
- [.../docs/auth-md](https://workos.com/docs/auth-md) — file format and required sections
- [.../docs/flows/verified](https://workos.com/docs/flows/verified) and [.../docs/flows/claimed](https://workos.com/docs/flows/claimed) — the two flows
- [.../docs/apps](https://workos.com/docs/apps) — endpoints, metadata JSON, verification, matching, rate limits, security checklist
- [.../docs/agent-providers](https://workos.com/docs/agent-providers) — ID-JAG header and claims
- [github.com/workos/auth.md](https://github.com/workos/auth.md) — reference implementation and the canonical AUTH.md (request/response shapes, error table)
- [WorkOS blog: Agent Registration with auth.md](#) — design rationale
- [MarkTechPost, 25 May 2026](#) — release coverage
- [IETF draft: Identity Assertion JWT Authorization Grant \(ID-JAG\)](#)